	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor: René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 8

Integrante(s): 120002850 323064039

323071378 323214636

426059576 426058744

Semestre: 2027-1

Fecha de entrega: 23 de octubre de 2026

Obervaciones:

CALIFICACIÓN:

Índice

1. Introducción **2**

1.1. Planteamiento del problema 2

1.2. Objetivos 2

2. Marco Teórico **2**

3. Desarrollo **4**

3.1. Investigación y análisis de la consulta 4

3.2. Interfaz gráfica de login 10

3.3. Contrato de huella de carbono 12

3.4. Clase Edificio 13

3.5. Clase Auto 14

3.6. Clase Bicicleta 16

3.7. Clase principal y polimorfismo 17

4. Resultados **19**

4.1. Casos y resultados numéricos 19

4.2. Ejecución desde clases y desde JAR 21

4.3. Documentación y empaquetado 22

5. Conclusiones **23**

1. Introducción

1.1. Planteamiento del problema

La práctica solicita resolver dos necesidades relacionadas con la programación orientada a objetos: la primera consiste en plantear una interfaz gráfica elemental para capturar un usuario y una contraseña, la segunda requiere representar tres entidades distintas —un edificio, un automóvil y una bicicleta— mediante clases que implementen una misma interfaz llamada `HuellaCarbono`, cada objeto debe calcular su huella con los datos propios de su dominio y la clase principal debe recorrer una lista de referencias comunes para demostrar polimorfismo.

El ejercicio permite observar que una interfaz define un prototipo sin obligar a que todas las clases compartan la misma fórmula. El edificio relaciona consumo eléctrico y factor de emisión; el automóvil relaciona distancia, rendimiento y combustible; la bicicleta expresa que su operación no produce emisiones directas.

1.2. Objetivos

- Implementar y comprobar una solución básica en Java que use una interfaz, implementaciones concretas y polimorfismo.
- Crear las clases `Edificio`, `Auto` y `Bicicleta`
- Declarar el método `getHuellaCarbono()` y almacenar las instancias en un `ArrayList<HuellaCarbono>`

2. Marco Teórico

Una interfaz en Java declara un prototipo de operaciones que una clase concreta debe cumplir. La especificación del lenguaje describe que una clase puede implementar una o varias interfaces y que una referencia al tipo de la interfaz puede utilizarse para invocar operaciones definidas por ese prototipo [1]. En esta práctica, `HuellaCarbono` solo declara `getHuellaCarbono()`, mientras que cada clase determina la forma de obtener el valor. La selección de la implementación ocurre durante la ejecución, lo que constituye el polimorfismo utilizado por el recorrido de la lista.

El encapsulamiento concentra los datos de cada entidad dentro de la clase que conoce su significado, por ello, los consumos, distancias y factores se mantienen como atributos privados, mientras que los métodos públicos ofrecen consultas y el cálculo requerido. La solución evita herencia porque las tres entidades no forman una jerarquía natural: comparten el prototipo de huella, pero no una estructura de estado común.

La interfaz gráfica se construye con Java Swing. Un `JFrame` funciona como ventana; `JTextField` captura el usuario; `JPasswordField` proporciona un componente apropiado para una contraseña; `JButton` inicia la acción; y `JLabel` comunica el resultado. La documentación de Oracle presenta estos componentes como parte de la biblioteca de interfaces gráficas de Java[2].

La huella de carbono depende del alcance y de los factores seleccionados, los inventarios oficiales separan fuentes y unidades, por lo que un factor eléctrico debe interpretarse según la región y el periodo analizado [3]. Para el ejercicio se emplean dos factores: 0.40 kg CO_2e/kWh para electricidad y 2.31 kg CO_2e/L para gasolina. No se presentan como valores universales; solo permiten comprobar una transformación aritmética estable. [4].

3. Desarrollo

3.1. Investigación y análisis de la consulta

Las diferencias se resumen en el enfoque: por un lado se ofrece una solución práctica y técnica mediante código funcional en Java para implementar la ventana directamente; y por el otro, se presenta una explicación conceptual y general, resumiendo de forma breve la estructura visual y los elementos básicos que componen cualquier pantalla de login, sin depender de un lenguaje de programación específico.

La primera reserva corresponde a Alan.

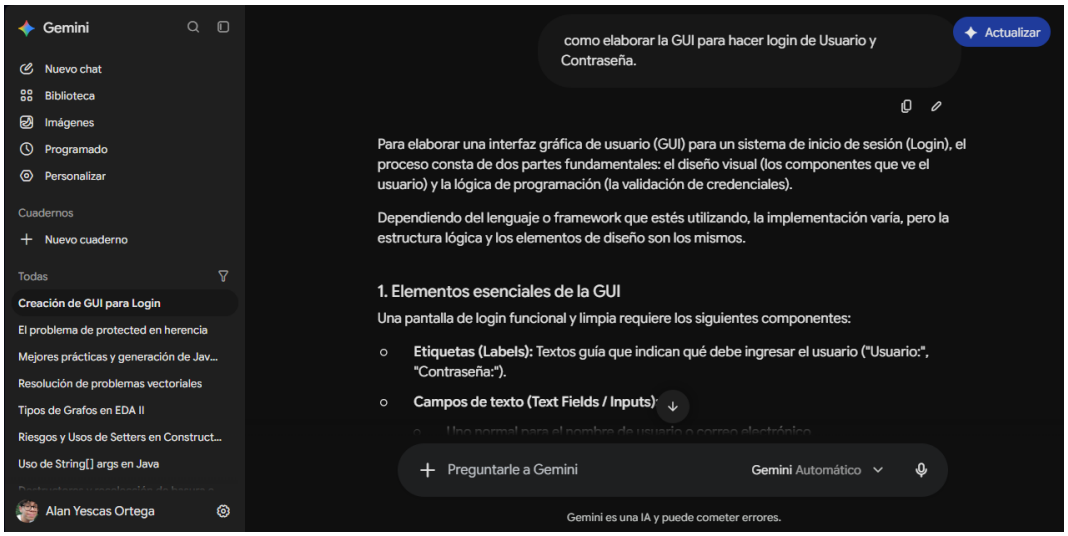


Figura 3.1: Investigación 1: Alan.

La segunda reserva corresponde a Zoé.

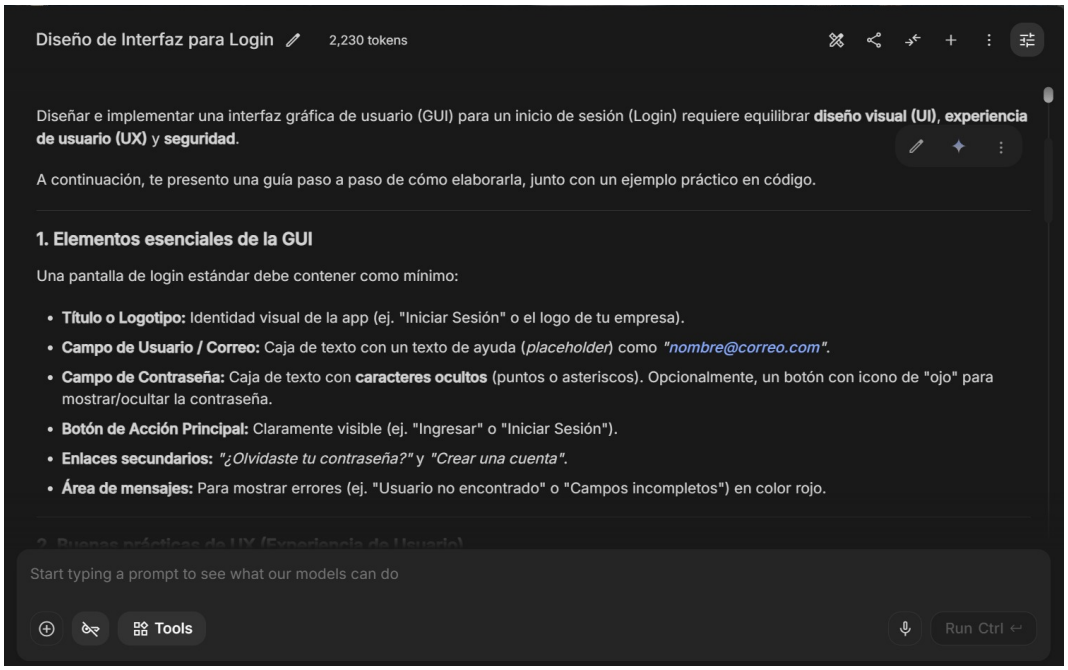


Figura 3.2: Investigación 2: Zoé.

La tercera reserva corresponde a Luis.



Figura 3.3: Investigación 3: Luis.

La cuarta reserva corresponde a Joseph.



User • 7:43 PM

Cómo elaborar la GUI para hacer login de Usuario y Contraseña. En Java. De forma breve y resumida.

Model • 7:43 PM

Para crear una GUI de login rápida y funcional en Java, la forma estándar y sin librerías externas es usando **Swing**.

Aquí tienes el código completo, funcional y directo:

Java

```java
import javax.swing.*;
import java.awt.*;

public class LoginApp {
    public static void main(String[] args) {
        // 1. Crear la ventana
        JFrame frame = new JFrame("Iniciar Sesión");
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new GridLayout(3, 2, 5, 5)); // Cuadrícula de 3 filas, 2 columnas

        // 2. Crear los componentes
        JLabel userLabel = new JLabel("Usuario:");
        JTextField userText = new JTextField();

        JLabel passLabel = new JLabel("Contraseña:");
        JPasswordField passText = new JPasswordField(); // Oculta los caracteres

        JButton loginButton = new JButton("Ingresar");

        // 3. Acción del botón
        loginButton.addActionListener(e -> {
            String usuario = userText.getText();
            String password = new String(passText.getPassword());

            // Validación simple de prueba
            if (usuario.equals("admin") && password.equals("1234")) {
                JOptionPane.showMessageDialog(frame, "¡Acceso concedido!");
            } else {
                JOptionPane.showMessageDialog(frame, "Usuario o contraseña incorrectos", "Error", JOptionPane.ERROR_MESSAGE);
            }
        });

        // 4. Agregar componentes a la ventana
        frame.add(userLabel);
        frame.add(userText);
        frame.add(passLabel);
        frame.add(passText);
        frame.add(new JLabel("")); // Espacio vacío para alinear el botón
        frame.add(loginButton);

        // 5. Centrar y mostrar
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

Figura 3.4: Investigación 4: Joseph.

La quinta reserva corresponde a Hanniel.

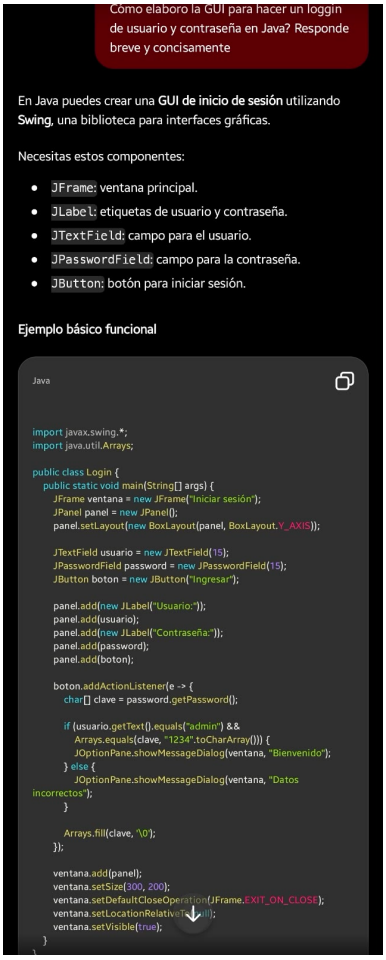


Figura 3.5: Investigación 5: Hanniel.

La sexta reserva corresponde a Alex.

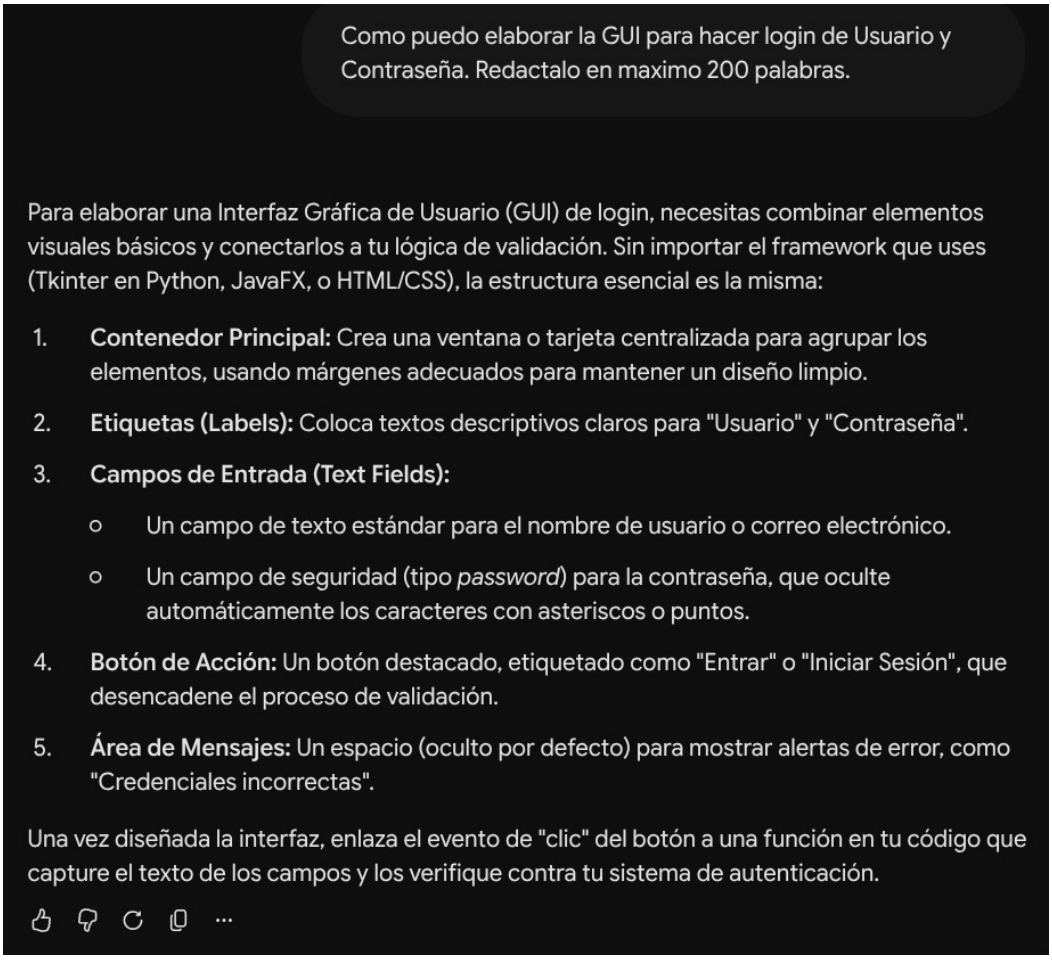


Figura 3.6: Investigación 6: Alex.

3.2. Interfaz gráfica de login

La clase `LoginGUI` extiende `JFrame` e implementa `ActionListener`, este contiene un campo de usuario, un campo de contraseña, un botón y una etiqueta de estado, la validación es deliberadamente pequeña y demostrativa: compara con el usuario `usuario` y la contraseña `poo8`.

Esta decisión permite evidenciar el flujo de entrada, validación y respuesta sin convertir el ejercicio en un sistema de autenticación real.

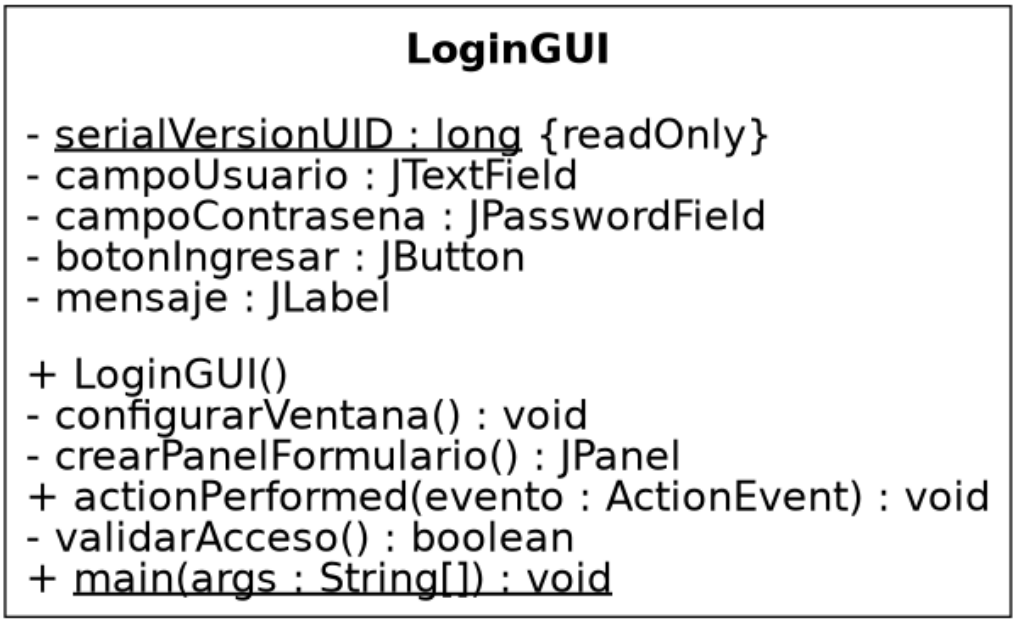


Figura 3.7: Diagrama UML de la clase LoginGUI.

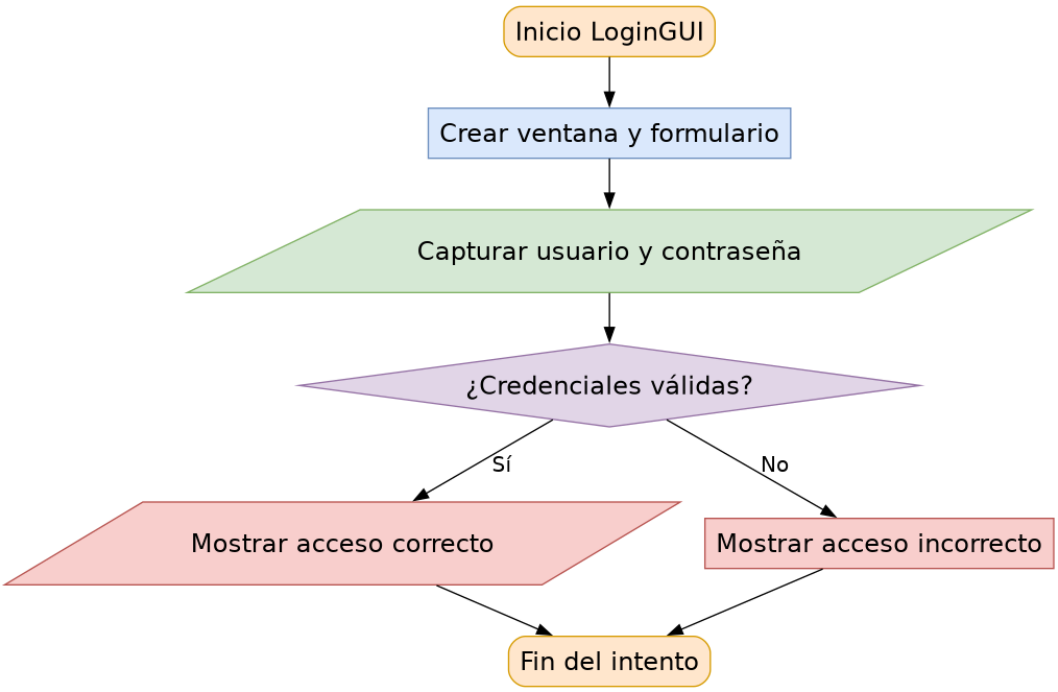


Figura 3.8: Flujo de captura y validación del login.

3.3. Contrato de huella de carbono

La interfaz `HuellaCarbono` contiene un solo método: `double getHuellaCarbono()`. Al mantener el contrato mínimo, el programa principal puede trabajar con referencias del tipo de la interfaz sin preguntar qué clase concreta calcula el valor, el diagrama UML conserva el compartimento de atributos vacío porque la interfaz no declara estado, y el flujo representa la delegación hacia la clase concreta.

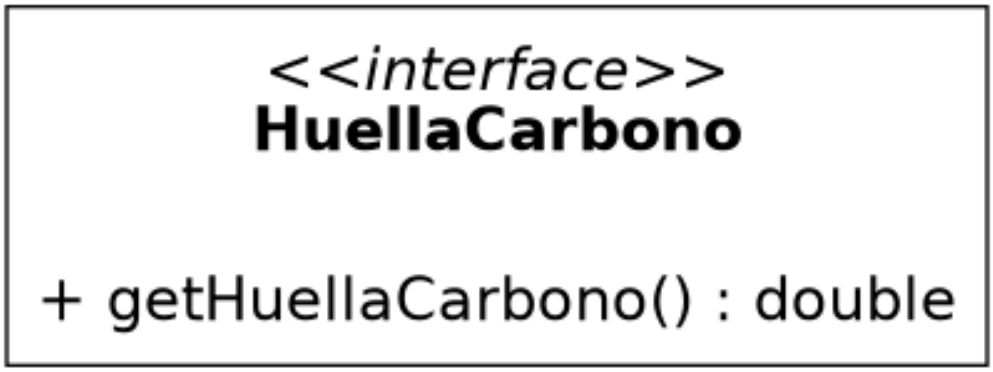


Figura 3.9: Diagrama UML de la interfaz HuellaCarbono.

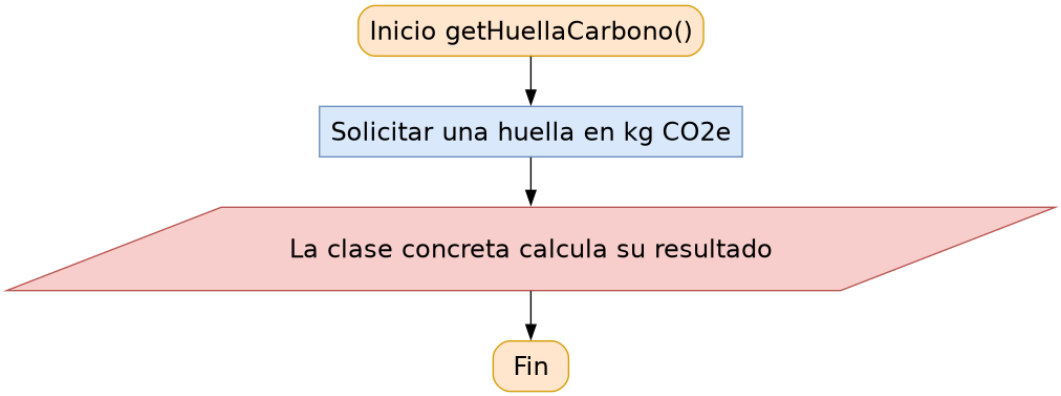


Figura 3.10: Flujo conceptual de la operación getHuellaCarbono.

3.4. Clase Edificio

`Edificio` identifica una construcción por su nombre, consumo anual de electricidad y factor de emisión, la fórmula implementada es $H = C \times F$, donde C se mide en kilowatt-hora y F en kilogramos de CO_2e por kilowatt-hora.

El comportamiento es exclusivo de esta clase porque las otras entidades no utilizan un consumo eléctrico anual.

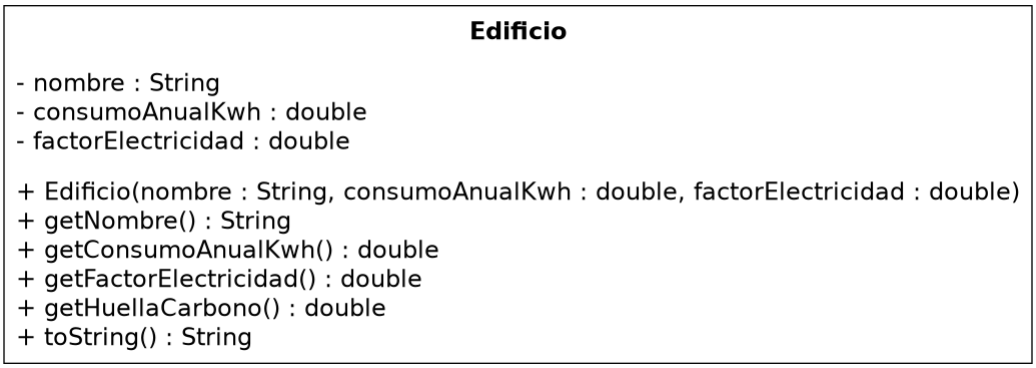


Figura 3.11: Diagrama UML de la clase Edificio.

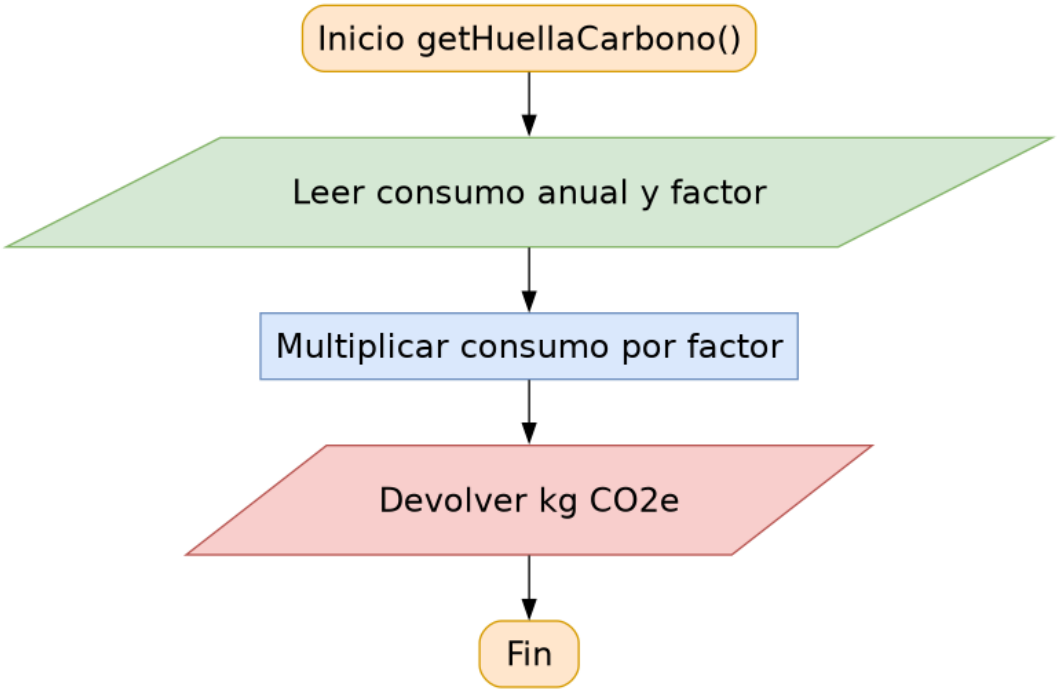


Figura 3.12: Flujo del cálculo de huella de Edificio.

3.5. Clase Auto

`Auto` identifica un modelo y calcula el combustible utilizado con $L = (K \times R)/100$, donde K es la distancia en kilómetros y R es el rendimiento expresado en litros por cada cien kilómetros, posteriormente obtiene la huella con $H = L \times F_g$, usando el factor de gasolina F_g. La separación en dos operaciones hace visible la conversión intermedia y facilita comprobar el resultado.

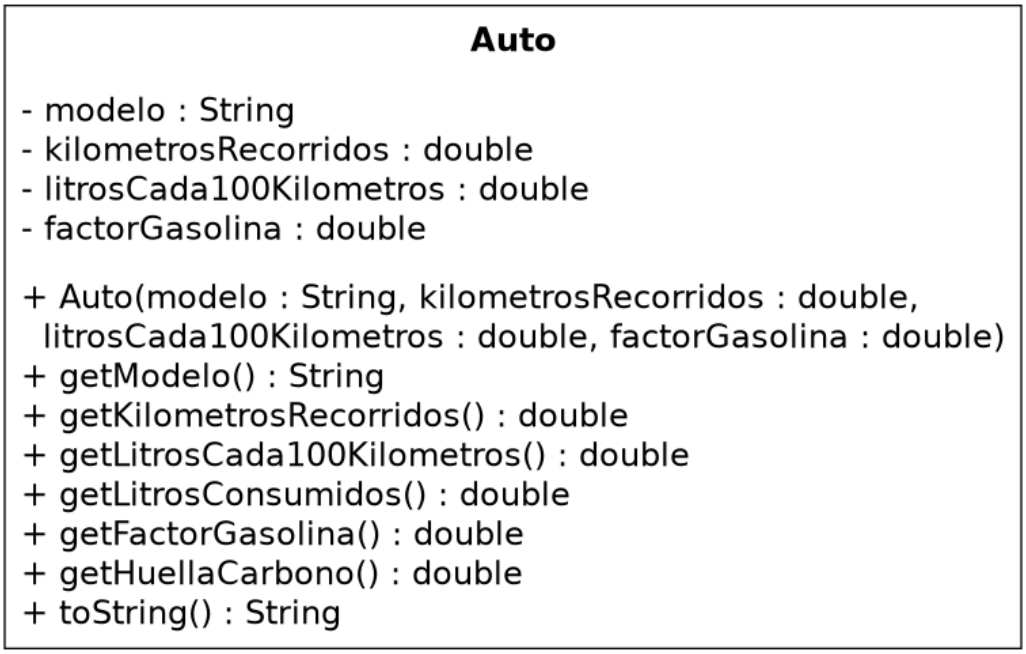


Figura 3.13: Diagrama UML de la clase Auto.

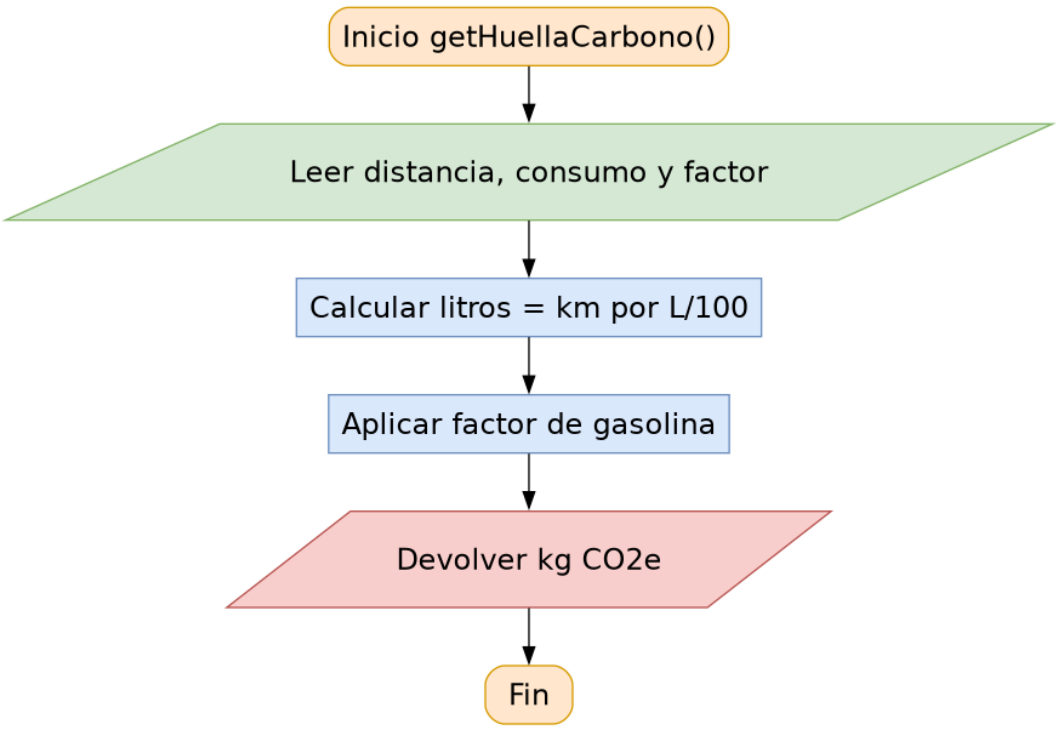


Figura 3.14: Flujo del cálculo de huella de Auto.

3.6. Clase Bicicleta

`Bicicleta` conserva un tipo y una distancia recorrida, el alcance del modelo se limita a emisiones operativas directas; por ello, el factor se fija en cero y la fórmula $H = K \times 0$ devuelve cero, pero no se afirma que la fabricación, alimentación, mantenimiento o transporte sean nulos, porque esos aspectos no forman parte del enunciado ni del cálculo implementado.

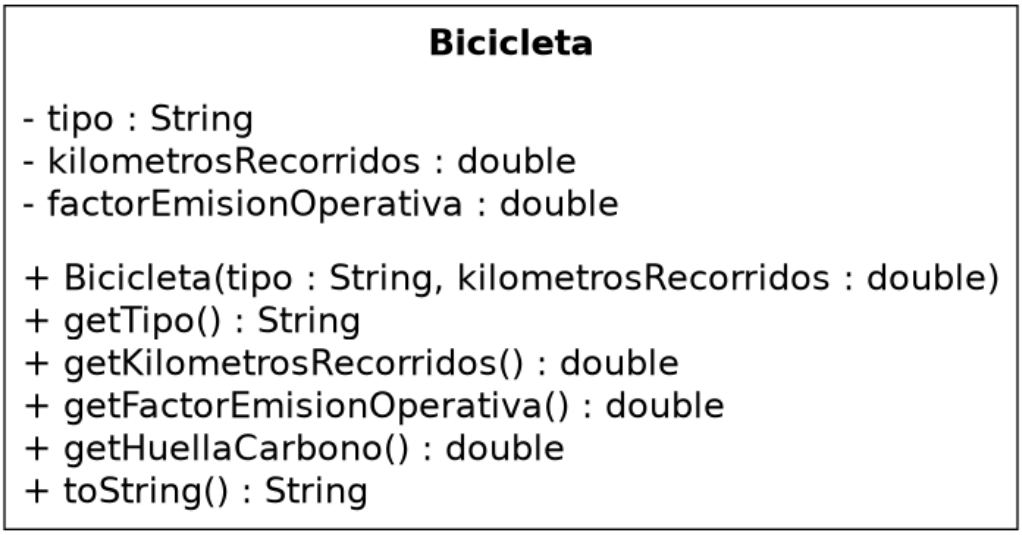


Figura 3.15: Diagrama UML de la clase Bicicleta.

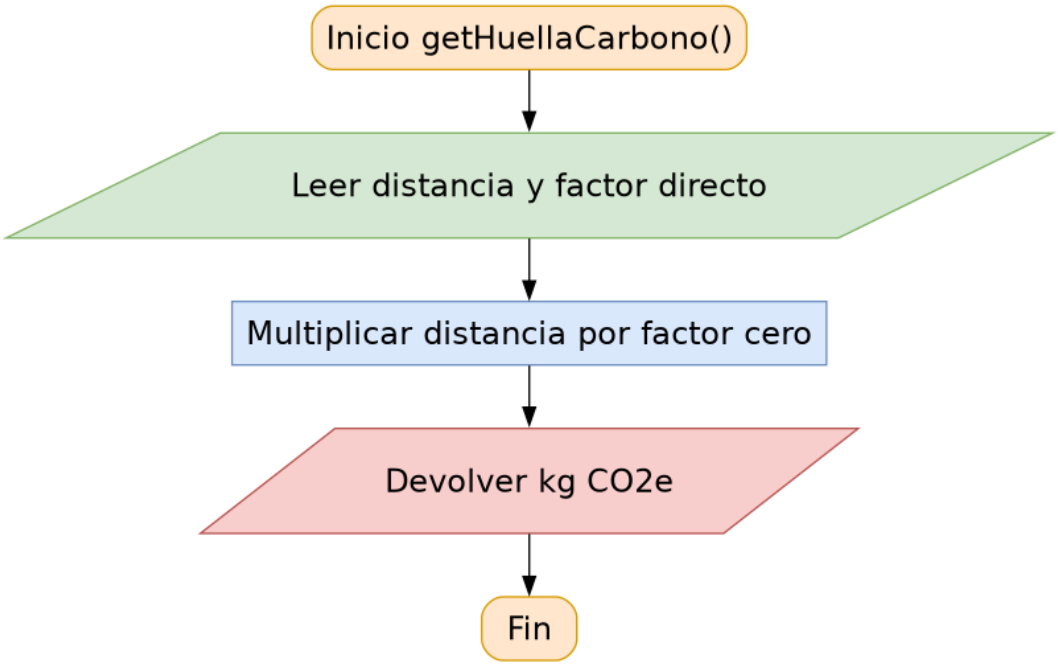


Figura 3.16: Flujo del cálculo de huella operativa de Bicicleta.

3.7. Clase principal y polimorfismo

La clase `Main` crea las tres instancias exigidas y agrega también un edificio con consumo cero como frontera pertinente, asi mismo, las cuatro referencias se guardan en un `ArrayList<HuellaCarbono>`, en él ciclo solo conoce el contrato de la interfaz: imprime la identificación del objeto y llama a `getHuellaCarbono()`, selecciona la fórmula de `Edificio`, `Auto` o `Bicicleta` sin condicionales por tipo.

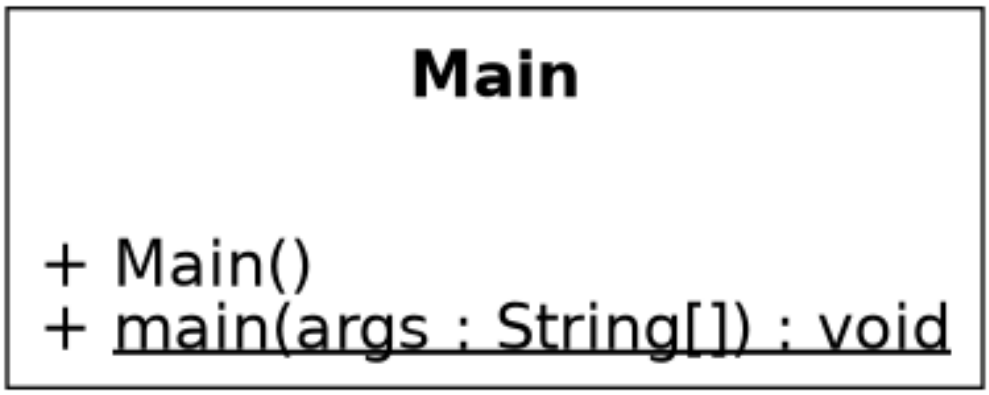


Figura 3.17: Diagrama UML de la clase principal Main.

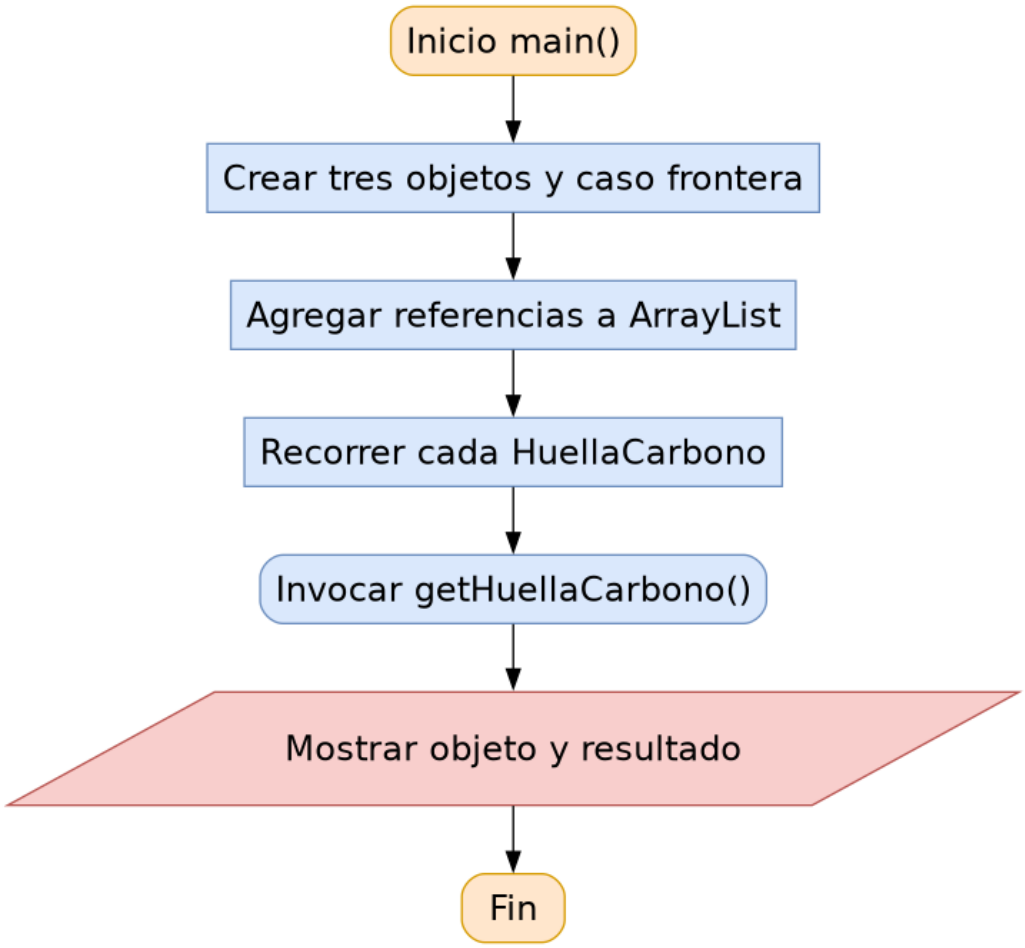


Figura 3.18: Flujo de creación, recorrido e invocación polimórfica.

4. Resultados

4.1. Casos y resultados numéricos

Los resultados esperados se calcularon antes de ejecutar el programa con las fórmulas descritas en el desarrollo.

El edificio principal usa $2400 \times 0.40 = 960.00$ kg CO_2e.

El automóvil usa $480 \times 7/100 = 33.60$ litros y después $33.60 \times 2.31 = 77.62$ kg CO_2e, redondeado a dos decimales.

La bicicleta produce cero emisiones operativas directas en el modelo.

Finalmente, el edificio frontera permite comprobar que un consumo nulo no introduce una huella espuria.

Tabla 1: Comparación entre el esperado calculado y el observado ejecutado.

Objeto	**Entrada principal**	**Esperado**	**Observado**	**Estado**
Edificio Escuela	2400 kWh, factor 0.40	960.00	960.00	Exitoso
Auto Sedán	480 km, 7.0 L/100 km, factor 2.31	77.62	77.62	Exitoso
Bicicleta urbana	12 km, factor operativo 0.00	0.00	0.00	Exitoso
Edificio frontera	0 kWh, factor 0.40	0.00	0.00	Exitoso

La ejecución observada también confirma la identificación de cada instancia y el recorrido de la lista mediante la interfaz. No se comparó contra una salida inventada: las cifras anteriores se obtuvieron de la salida real de la clase compilada y del JAR, cuyos registros visuales se incluyen en las figuras siguientes.

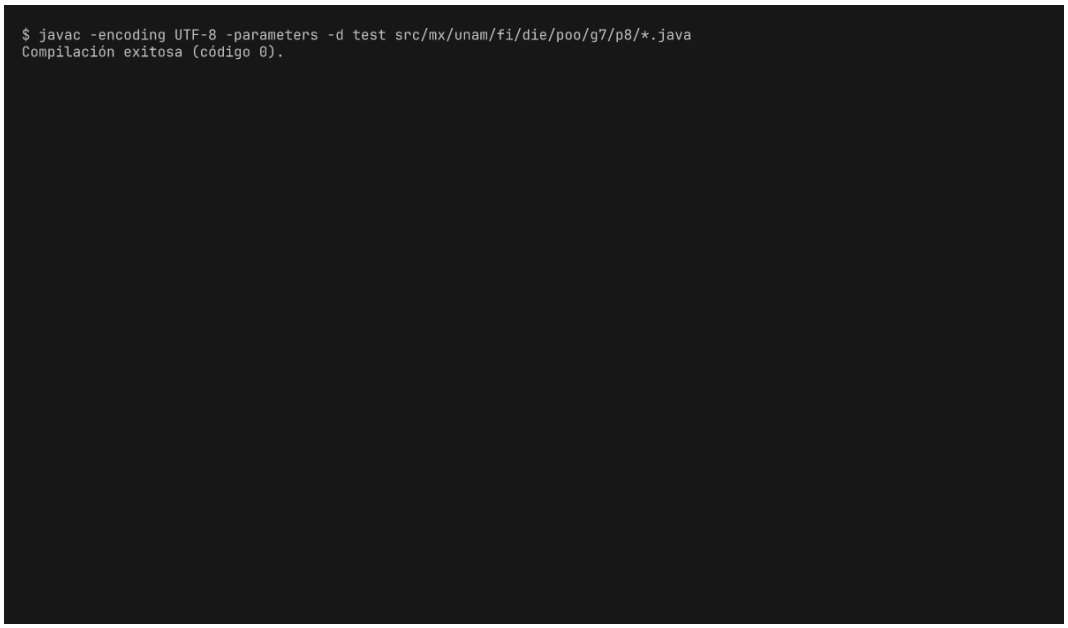


Figura 4.1: Compilación limpia de las seis fuentes Java hacia la carpeta test.

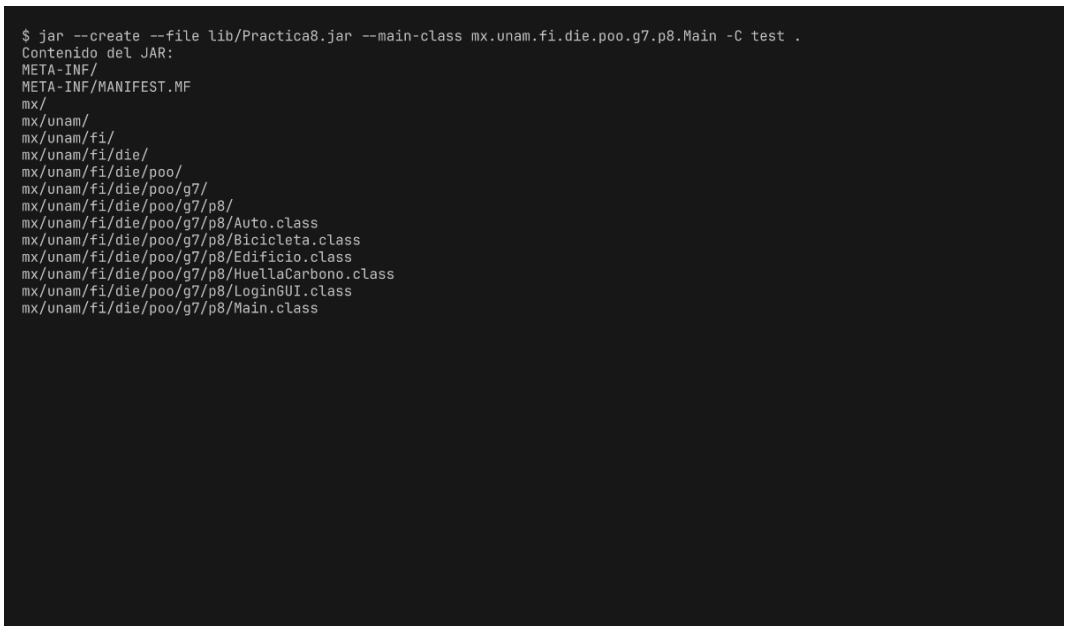


Figura 4.2: Creación y contenido del `Practica8.jar`.

4.2. Ejecución desde clases y desde JAR

La clase principal se ejecutó directamente desde la carpeta de compilados y posteriormente desde el archivo JAR con manifiesto de entrada, ambas modalidades produjeron las mismas cuatro huellas: 960.00, 77.62, 0.00 y 0.00 kg CO_2e, a su vez, el JAR contiene `Main.class`, la interfaz, las tres implementaciones de huella y la clase de GUI, por lo que no incorpora clases de prácticas anteriores.

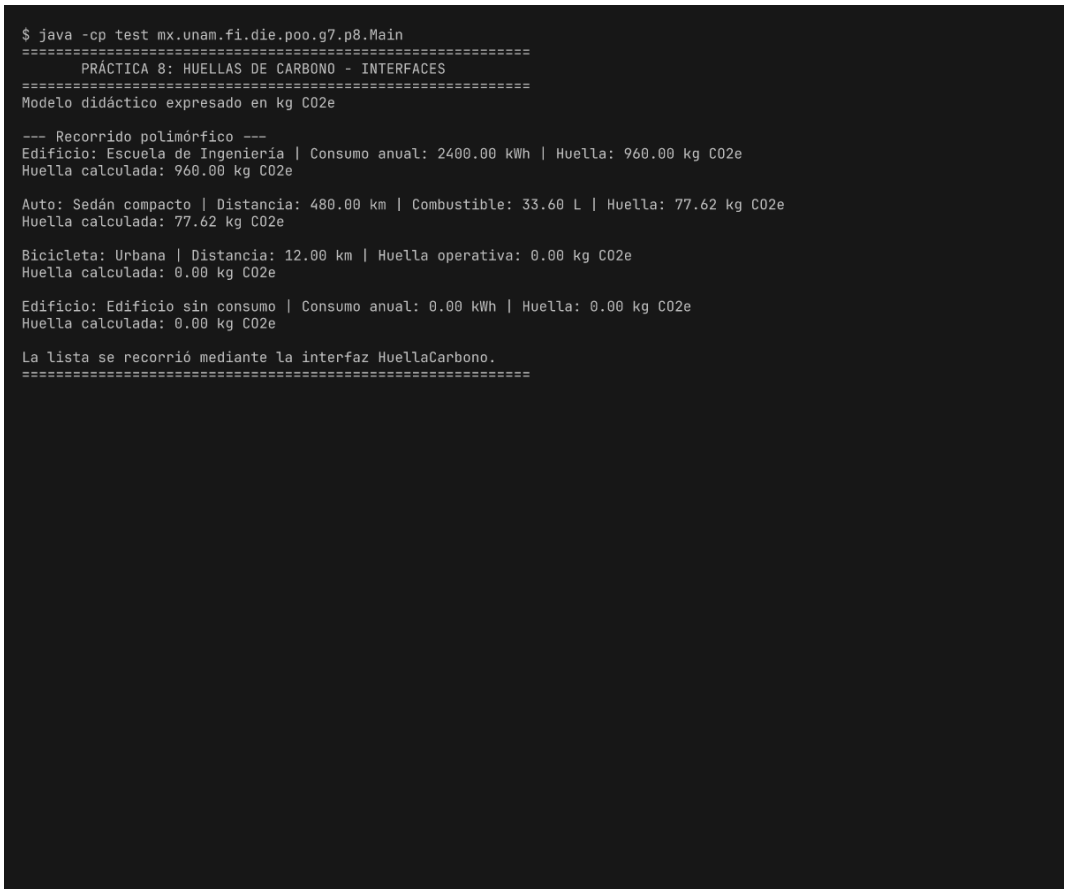


Figura 4.3: Salida observada al ejecutar Main desde la carpeta de compilados.

La compilación de `LoginGUI` fue exitosa y la documentación se generó de forma independiente, por lo que no se declara una ejecución visual observada de la ventana porque el entorno de trabajo no dispone de un servidor gráfico; la verificación disponible para este componente es compilación, firmas, Javadoc y revisión estática del flujo. Esta limitación no afecta la ejecución determinista de la parte de consola.

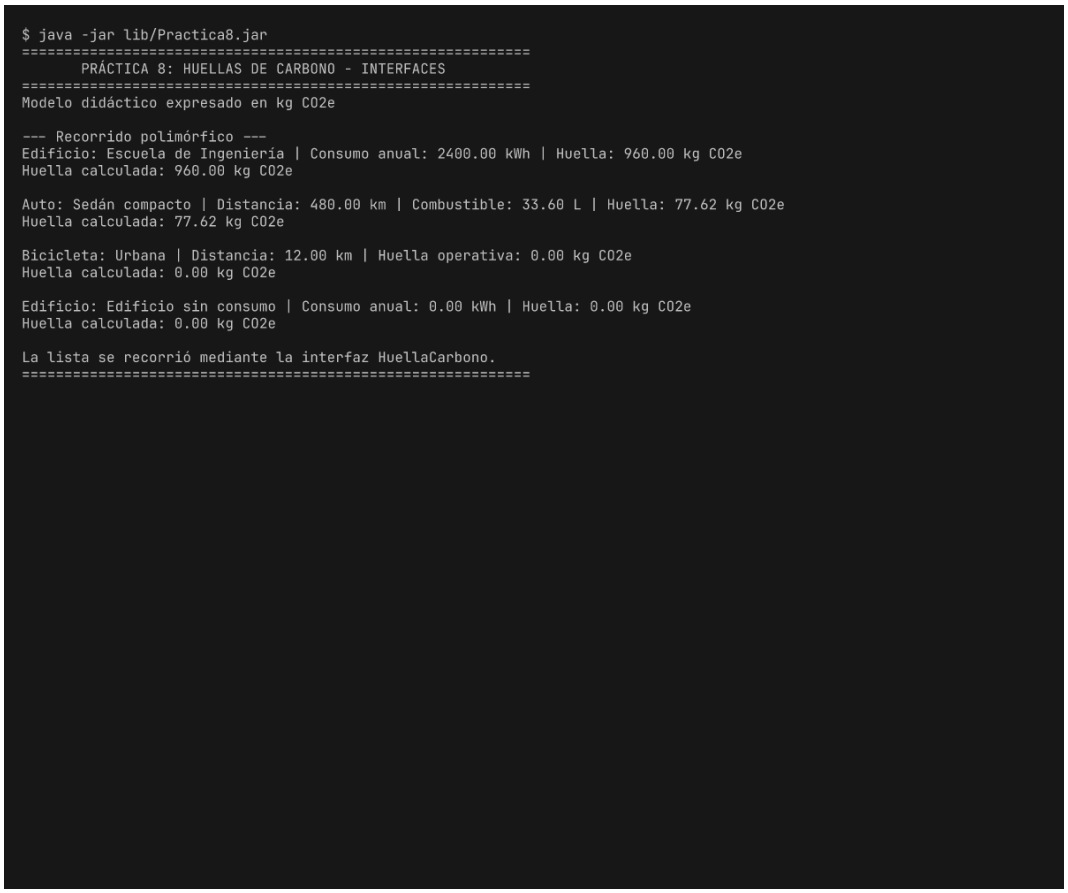


Figura 4.4: Salida observada al ejecutar el trabajo completo desde el JAR.

4.3. Documentación y empaquetado

Se generó una salida independiente de Javadoc para `HuellaCarbono`, `Edificio`, `Auto`, `Bicicleta`, `Main` y `LoginGUI`, cada una en su directorio bajo `doc/Javadoc`, también se verificó que `test/` contiene solo los compilados finales y que `lib/` contiene un solo archivo JAR del trabajo.

5. Conclusiones

La interfaz `HuellaCarbono` permitió unificar tres comportamientos que no comparten una jerarquía de datos, cada clase conserva sus atributos y su fórmula, mientras que `Main` utiliza una referencia común y una lista homogénea. Esta separación muestra una aplicación concreta del polimorfismo: el código que recorre la colección no necesita conocer la clase exacta de cada elemento.

Los resultados numéricos coincidieron con los esperados calculados antes de ejecutar, los casos de la frontera de consumo cero confirmó el comportamiento mínimo del modelo, y las ejecuciones desde classpath y JAR conservaron la misma salida, de la misma manera los factores de emisión se mantuvieron como supuestos didácticos explícitos, porque una evaluación real tendría que seleccionar factores apropiados para una región, periodo y alcance definidos.

La GUI de login completó el flujo académico de captura, validación y respuesta mediante componentes básicos de Swing y su validación no pretende sustituir autenticación segura; su función es demostrar la interacción gráfica solicitada.

En conjunto, la práctica relaciona contrato, implementaciones concretas, documentación y empaquetado reproducible sin agregar una arquitectura innecesaria.

Bibliografía

[1] Oracle, *The Java Language Specification, Java SE 25 Edition: Interfaces*, Oracle Corporation, 2025.

[2] Oracle, *Creating a GUI With Swing*, The Java Tutorials, Oracle Corporation.

[3] U.S. Environmental Protection Agency, *Emissions & Generation Resource Integrated Database (eGRID)*, EPA, 2024.

[4] U.S. Environmental Protection Agency, *Emission Factors for Greenhouse Gas Inventories*, EPA, 2024.